

Lab Report

Experiment No: 01

Experiment Title: Study of Basic Python Syntax, Data Types, Operators and Conditional Statements

Name: Arif

Student ID: E251043

Department: Computer and Communication Engineering (CCE), IIUC

1. Objective

- 1. To learn the fundamental syntax rules of Python programming.
- 2. To identify and use Python's built-in data types and operators.
- 3. To control program flow using conditional statements (if, elif, else, match).

2. Basic Syntax

2.1 Variable Assignment and Type Checking

```
university = 'IIUC'  
print(university)  
print(type(university))
```

Output:

```
IIUC  
<class 'str'>
```

2.2 f-string and Multiline String

```
course = 'CCE'  
print(f'Department: {course}')
```

Multiline string

```
print('''  
Row 1  
Row 2  
Row 3  
''')
```

Output:

```
Department: CCE  
Row 1  
Row 2  
Row 3
```

3. Data Types

3.1 Number (int)

```
reg_no = 43  
print(reg_no)  
print(type(reg_no))
```

Output:

```
43  
<class 'int'>
```

3.2 String

```
batch = 'Fall25'
```

```
print(batch)
print(type(batch))
```

Output:

```
Fall25
<class 'str'>
```

3.3 List (mutable)

```
languages = ['Python', 'C']
print(languages)
print(type(languages))
languages[1] = 'Java' # updating is allowed
print(languages)
```

Output:

```
['Python', 'C']
<class 'list'>
['Python', 'Java']
```

3.4 Tuple (immutable)

```
languages_tuple = ('Python', 'Java')
print(languages_tuple)
print(type(languages_tuple))
```

Output:

```
('Python', 'Java')
<class 'tuple'>
```

3.5 Dictionary

```
profile = {'name': 'Arif', 'dept': 'CCE'}
print(profile)
print(type(profile))
```

Output:

```
{'name': 'Arif', 'dept': 'CCE'}
<class 'dict'>
```

3.6 Set

```
attendance = {1, 2, 3, 2}
print(attendance)
print(type(attendance))
```

Output:

```
{1, 2, 3}
<class 'set'>
```

3.7 Boolean

```
is_enrolled = True
print(type(is_enrolled))
```

Output:

```
<class 'bool'>
```

3.8 NoneType

```
result = None
print(type(result))
```

Output:

```
<class 'NoneType'>
```

4. String Operations

4.1 String Reverse

```
text = 'Coding'  
print(text[::-1])
```

Output:

gnidoC

4.2 String Slicing

```
phrase = 'Practice makes perfect'  
print(phrase[0:8])
```

Output:

Practice

5. Operators

5.1 Arithmetic Operators

```
a = 10  
b = 4  
x = a / b    # float division  
print(x)
```

```
a = 2  
b = 6  
x = a ** b   # exponential  
print(x)
```

Output:

*2.5
64*

5.2 Bitwise Operators

```
a = 20  
b = 3  
x = (a >> b)  
print(x)
```

Output:

2

5.3 Comparison Operators

```
a = 6  
b = 6  
print(a > b)  
print(a == b)  
print(a != b)
```

Output:

*False
True
False*

5.4 Logical Operators (and, or, not)

```
a = False  
b = False  
print(a and b)
```

```
a = True
b = False
print(a or b)
```

```
a = False
print(not a)
```

Output:

```
False
True
True
```

5.5 Membership Operators (in, not in)

```
languages = ['Python', 'Java']
print('Java' in languages)
print('C++' not in languages)
```

Output:

```
True
True
```

6. Conditional Statements

6.1 if - else

```
attendance_percent = 78
if attendance_percent >= 75:
    print('Eligible for Exam')
else:
    print('Not Eligible')
```

Output:

```
Eligible for Exam
```

6.2 if - elif - else

```
semester = 2
if semester == 1:
    print('First Semester')
elif semester == 2:
    print('Second Semester')
else:
    print('Other Semester')
```

Output:

```
Second Semester
```

6.3 Nested if - else

```
cgpa = 3.5
has_backlog = False
if cgpa >= 3.0:
    if has_backlog:
        print('Hold Certificate')
    else:
        print('Certificate Approved')
else:
    print('Not Approved')
```

Output:

```
Certificate Approved
```

6.4 match - case

```
semester = 2
match semester:
    case 1:
        print('First Semester')
    case 2:
        print('Second Semester')
    case _:
        print('Other Semester')
```

Output:

Second Semester

6.5 Ternary (Conditional) Expression

```
attendance_percent = 60
status = 'Allowed' if attendance_percent >= 75 else 'Not Allowed'
print(status)
```

Output:

Not Allowed

7. Discussion

Through this experiment, the fundamental syntax of Python, its core data types, the main categories of operators, and conditional control structures were explored and tested in Google Colab. It was confirmed that Python is a case-sensitive, dynamically typed language, where the interpreter automatically assigns a variable's type based on its value. Comparing list and tuple showed that a list can be edited after it is created, while a tuple stays fixed and unchangeable. Logical and membership operators returned Boolean results, which made them well suited for decision-based logic. Among the conditional structures, the match-case statement acted like a switch-case but did not need a break statement to prevent fall-through. In summary, the experiment strengthened the understanding of core Python programming concepts.